

ניווט

בטאיוואן יש צפיפות גדולה של הרים גבוהים, עם למעלה מ-250 פסגות מעל 3,000 מטר. א-מינג (A-Ming) מתכנן להקים רשתות של אומגות רובוטיות למשלוחים ברכס ההרים המרכזי של טאיוואן, כדי להעביר חבילות בין הכפרים המרוחקים.

בתוכניתו של א-מינג, כל רשת מורכבת מ- N צמתים, הממוספרים מ-0 עד $N - 1$. מבין צמתים אלו, בדיוק $K = 6$ הם תחנות כוח, ושאר $N - K$ הצמתים הם כולם צמתי כפרים. הצמתים מחוברים על ידי $N - 1$ כבלים דו-כיווניים, הממוספרים מ-0 עד $N - 2$. עבור כל i ($0 \leq i \leq N - 2$), כבל i מחבר את הצמתים $U[i]$ ו- $V[i]$. כל כבל מחבר שני צמתים שונים, וכל זוג צמתים מחובר לכל היותר על ידי כבל אחד. מובטח כי ניתן לעבור בין כל זוג צמתים באמצעות הכבלים.

המרחק בין צמתים a ו- b מסומן ב- $d(a, b)$ ומוגדר כדלקמן.

- אם $a = b$ אזי $d(a, b) = 0$.
- אחרת, $d(a, b)$ הוא המספר המינימלי של כבלים הנדרשים כדי לעבור מ- a ל- b .

אנו אומרים שמקדם ההסתעפות (branching factor) של הרשת הוא לפחות δ אם כל צומת ברשת מחובר בדיוק לכבל 1 או לפחות ל- δ כבלים. א-מינג מכיר מספר שלם חיובי B כך שמקדם ההסתעפות של הרשת הוא לפחות B .

א-מינג הכין 100 סוגים שונים של כבלים, הממוספרים 0, 1, ..., 99. כל כבל ברשת ייבנה מאחד מהסוגים הללו.

הרובוטים ינועו לאורך כבלי האומגה כדי לבצע משימות משלוח. א-מינג מעוניין להתקין תוכנת ניווט שתעזור לרובוטים לחזור לתחנות הכוח ולהטעין את עצמם. עם זאת, לרובוטים יש זיכרון נמוך במיוחד. לכן, בעת תכנון תוכנת הניווט, הרובוטים מנווטים אך ורק בהתבסס על מספר תחנות הכוח $K = 6$, מקדם ההסתעפות המובטח B , וסוגי הכבלים המחוברים למיקומו הנוכחי של הרובוט.

כאשר רובוט רוצה לנווט בצומת u המחובר לשני כבלים או יותר, הרובוט סורק תחילה את הכבלים המחוברים ל- u בסדר שרירותי. לאחר מכן, מסופקת לתוכנה רשימה המכילה את סוגי הכבלים לפי סדר זה. עבור כל אחד מהכבלים, התוכנה צריכה לספור את מספר תחנות הכוח שעבורן תנועה לאורך הכבל מקטינה את המרחק בין הרובוט לתחנה.

באופן ספציפי, יהיו $v_1, \dots, v_k$ ($k \geq 2$) הצמתים המחוברים ישירות ל- u על ידי כבל, ו- c_i ($1 \leq i \leq k$) יהיה סוג הכבל המחבר את צומת u ו- v_i . התוכנה מקבלת אז את B, K , ואת הרשימה $c_1, c_2, \dots, c_k$. עבור כל i ($1 \leq i \leq k$), התוכנה צריכה לקבוע את מספר תחנות הכוח p כך ש- $d(v_i, p) < d(u, p)$.

המשימה שלכם היא לתכנן אסטרטגיה להקצאת סוגי הכבלים ולעצב את תוכנת הניווט. הציון של הפתרון שלכם תלוי במספר הסוגים השונים של כבלים שבהם נעשה שימוש (ראה פרק תת-משימות וניקוד לפרטים).

פרטי מימוש

עליכם לממש שתי פונקציות, האחת להקצאת סוגי הכבלים והשנייה עבור תוכנת הרובוטים.

הפונקציה שעליכם לממש כדי להקצות את סוגי הכבלים היא:

```
std::vector<int> construct_network(int N, int K, int B,  
                                  std::vector<int> U, std::vector<int> V,  
                                  std::vector<int> P)
```

- N : מספר הצמתים.
- K : מספר תחנות הכוח.
- B : מקדם ההסתעפות המינימלי המובטח של הרשת.
- U, V : מערכים באורך $N - 1$ המתארים את חיבורי הכבלים.
- P : מערך באורך $K = 6$ המתאר את צמתי תחנות הכוח.
- פונקציה זו נקראת לכל היותר 10 000 פעמים עבור כל מקרה בדיקה (test case).

פונקציה זו צריכה להחזיר מערך T באורך $N - 1$:

- כבל מסוג $T[i]$ משמש לחיבור הצמתים $U[i]$ ו- $V[i]$.
- כל איבר $T[i]$ חייב לקיים $0 \leq T[i] < 100$.

הפונקציה שעליכם לממש עבור תוכנת הרובוטים היא:

```
std::vector<int> navigate(int K, int B, std::vector<int> C)
```

- K : מספר תחנות הכוח.
- B : מקדם ההסתעפות המינימלי המובטח של הרשת.
- C : רשימת הסוגים של כל הכבלים המחוברים לצומת כפר מסוים, בסדר שרירותי.
- מובטח כי האורך של C הוא לפחות B .
- פונקציה זו נקראת לכל היותר 100 000 פעמים עבור כל מקרה בדיקה.
- הפונקציה `navigate` אינה רשאית להסתמך על מבנה הרשת המקורי; בפרט, מערכת הבדיקה עשויה לשנות את הסדר של כל הקריאות ל-`navigate` בין הרשתות השונות שנבנו באותו מקרה בדיקה.

פונקציה זו צריכה להחזיר מערך D :

- אורך המערך D חייב להיות זהה לזה של המערך C .
- $D[i]$ חייב להיות מספר תחנות הכוח שעבורן תנועה לאורך הכבל המתאים ל- $C[i]$ מקטינה את המרחק בין הרובוט לתחנה.

שימו לב שבמערכת הבדיקה, הפונקציות `construct_network` ו-`navigate` נקראות בשני תהליכים (processes) נפרדים.

אילוצים

- $K = 6$.
- $K < N \leq 100\,000$.
- סכום כל ה- N ימים על פני כל הקריאות ל-`construct_network` אינו עולה על 100 000 בכל מקרה בדיקה.
- $0 \leq U[i] < V[i] < N$ עבור כל $0 \leq i \leq N - 2$.
- ניתן לעבור מכל צומת לכל צומת אחר באמצעות הכבלים.
- $0 \leq P[0] < P[1] < \dots < P[K - 1] < N$.
- סכום אורכי המעריך C על פני כל הקריאות ל-`navigate` אינו עולה על 200 000 בכל מקרה בדיקה.

תת-משימות וניקוד

תת-משימה	ציון	אילוצים נוספים
1	6	$B = 2, N = 7$
2	14	$B = 2, U[i] = i, V[i] = i + 1$ עבור כל i כך ש- $0 \leq i < N - 1$
3	20	$B = 5$
4	20	$B = 4$
5	40	$B = 2$

בכל אחד ממקרי הבדיקה, אם מתקיים לפחות אחד מהתנאים הבאים, אזי הציון של הפתרון שלך עבור אותו מקרה בדיקה יהיה 0 (ידווח כ-`Output isn't correct` ב-CMS):

- ערך החזרה של קריאה כלשהי ל-`construct_network` אינו תקין.
- ערך החזרה של קריאה כלשהי ל-`navigate` שגוי.

אחרת, יהי S המספר השלם הקטן ביותר שגדול מכל המספרים בכל מערך T שהוחזר מכל קריאה ל-`construct_network`. במילים אחרות, S הוא המספר השלם הקטן ביותר כך שכל הרשתות שנבנו משתמשות אך ורק בכבלים מסוג $0, 1, \dots, S - 1$.

הציון שלך עבור כל תת-משימה תלוי ב- S , כדלקמן:

תנאי	תת-משימה 1	תת-משימה 2	תת-משימות 3 ו-4	תת-משימה 5
$100 < S$	0	0	0	0
$16 \leq S \leq 100$	2	2	$12 - \log_2 S$	$24 - 2 \log_2 S$
$10 \leq S \leq 15$	2	4	$16 - 0.5S$	$32 - S$
$7 \leq S \leq 9$	2	6	$21 - S$	$42 - 2S$
$S = 6$	2	10	15	30
$S = 5$	6	14	17.5	40
$S \leq 4$	6	14	20	40

בפרט, בתת-משימות 1, 2 ו-5, תקבלו ניקוד מלא אם $S \leq 5$, ובתת-משימות 3 ו-4, תקבל ניקוד מלא אם $S \leq 4$.

דוגמה

נתבונן בתרחיש שבו $N = 9$, $K = 6$ ו- $B = 2$.

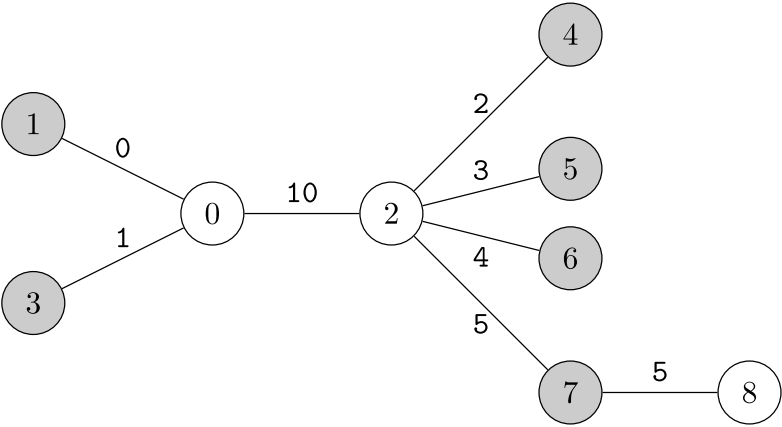
תוכנית הרשת מוגדרת על ידי $U = [0, 0, 0, 2, 2, 2, 2, 7]$, $V = [1, 2, 3, 4, 5, 6, 7, 8]$. תחנות הכוח הן $P = [1, 3, 4, 5, 6, 7]$. נתבונן בקריאה הבאה לתהליך הראשון:

```
construct_network(9, 6, 2, [0, 0, 0, 2, 2, 2, 2, 7],
                  [1, 2, 3, 4, 5, 6, 7, 8],
                  [1, 3, 4, 5, 6, 7])
```

נניח שא-מינג בונה את הרשת על ידי החזרת הערך:

```
[0, 10, 1, 2, 3, 4, 5, 5]
```

הרשת שנבנתה מומחשת להלן.



לאחר מכן, בתהליך השני, נניח שהרובוט צריך לנווט בצומת 0. הצומת 0 מחובר לצמתים 1, 2 ו-3. אם הרובוט קורא את סוג הכבל בסדר הזה, תתבצע הקריאה הבאה:

```
navigate(6, 2, [0, 10, 1])
```

בהתבסס על מבנה הרשת:

- לתחנת הכוח הראשונה, צומת 1, יש את המרחק הקצר ביותר לעצמה. בפרט, $0 = d(1, 1) < 1 = d(0, 1) < 2 = d(2, 1) = d(3, 1)$
- לתחנת הכוח השנייה, צומת 3, יש את המרחק הקצר ביותר לעצמה. בפרט, $0 = d(3, 3) < 1 = d(0, 3) < 2 = d(1, 3) = d(2, 3)$
- לתחנות הכוח האחרות, צמתים 4, 5, 6, ו-7, יש מרחקים קצרים יותר לצומת 2. בפרט, $1 = d(2, u) < 2 = d(1, u) < 3 = d(1, u) = d(3, u)$ כאשר $u = 4, 5, 6$ או 7.

מספר תחנות הכוח שהמרחק אליהן קטן לאחר מעבר בכבלים $(0, 1)$, $(0, 2)$, ו- $(0, 3)$ הוא 1, 4 ו-1, בהתאמה. לכן, הפונקציה צריכה להחזיר:

```
[1, 4, 1]
```

שים לב שניתן לקרוא ל-`navigate` גם עבור תמורות (permutations) שונות של C . לדוגמה, `navigate(6, 2, [1, 0, 10])` היא גם קריאה אפשרית עבור צומת 0. על הפונקציה להחזיר `[1, 1, 4]` בהתאם.

נניח שהרובוט צריך לנווט בצומת 2. תתבצע הקריאה הבאה:

```
navigate(6, 2, [10, 2, 3, 4, 5])
```

הפונקציה צריכה להחזיר:

```
[2, 1, 1, 1, 1]
```

ברשת זו, הפונקציה `navigate` לעולם לא תיקרא עבור צמתים אחרים, מכיוון ש:

- צמתים 1, 3, 4, 5, 6 ו-7 כולם תחנות כוח, ו-
- צומת 8 מחובר לכבל אחד בלבד.

במקרה בדיקה זה, סוגי הכבלים שנעשה בהם שימוש הם 0, 1, 2, 3, 4, 5 ו-10. לכן, יעשה שימוש ב- $S = 11$ לקביעת הציון.

חבילת הקבצים המצורפת למשימה זו מכילה שני קלטים ופליטים נוספים לדוגמה עבור תוכנת הבדיקה לדוגמה, בנוסף לדוגמה זו.

גריידר לדוגמה

הגריידר לדוגמה קורא גם ל-`construct_network` וגם ל-`navigate` באותו תהליך. בנוסף, כאשר תוכנת הבדיקה לדוגמה קוראת ל-`navigate`, סוגי הכבלים מופיעים בסדר שבו הכבלים המתאימים להם מופיעים בקלט. שתי ההתנהגויות הללו שונות מתוכנת הבדיקה האמיתית.

פורמט קלט:

```
T
(תרחיש 0)
(תרחיש 1)
...
(T-1 תרחיש)
```

כל תרחיש הוא בפורמט הבא:

```
N B
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
K
P[0] P[1] ... P[K-1]
Q
X[0] X[1] ... X[Q-1]
```

בכל תרחיש, הבודק לדוגמה יקרא ל-`navigate` Q פעמים, כאשר הקריאה ה- i תהיה עם המידע של צומת $X[i-1]$. שים לב שהבודק לדוגמה תומך בהגדרת K למספרים שלמים שאינם 6.

פורמט פלט:

אם אחד מהמערכים המוחזרים על ידי `construct_network` או `navigate` אינו תקין, הבודק לדוגמה עוצר ומדפיס את הסיבה. אחרת, הבודק לדוגמה מדפיס כל מערך D המוחזר מהקריאה ל-`navigate` בשורה חדשה.

לאחר שכל התרחישים עובדו, הבודק לדוגמה מדפיס שורה לפלט השגיאה התקני (standard error):

```
S = S'
```

שבו S' הוא המספר השלם הקטן ביותר שגדול מכל המספרים בכל מערך T שהוחזר מכל קריאה ל-`construct_network` במקרה הבדיקה.